

B+ Trees

Insertion :-

overflow :- when number of search key values exceed " $P-1$ "

Leaf Node :- split into 2 nodes

$\Rightarrow$ 1st Node contains $\lceil \frac{P-1}{2} \rceil$ values.

$\Rightarrow$ 2nd Node contains remaining values.

$\Rightarrow$ copy the smallest search key value of the 2nd node to the parent node.

Non-leaf Node :- split into 2 nodes

$\Rightarrow$ 1st Node contains $\lceil P/2 \rceil - 1$ keys

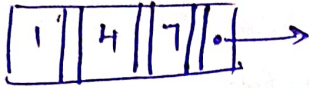
$\Rightarrow$ move the smallest of the remaining keys to the parent

$\Rightarrow$ 2nd Node contains Remaining keys.

Ex:- construct a B+ Tree for (1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42) with $P=4$ (order is 4) $\rightarrow$ keys are $(P-1) \rightarrow 3$

Sol:- construction of B+ Tree starts from leaf node

step 1:-



if we are trying to insert '10' key will become 4. then leaf overflow occurs.

step 2:-



$(1, 4) \rightarrow (7, 10, 17)$

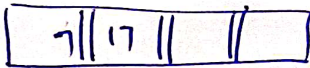
1st node contains $\lceil \frac{P-1}{2} \rceil$

2nd node contains remaining all values

move smallest value from 2nd node merge

it is done

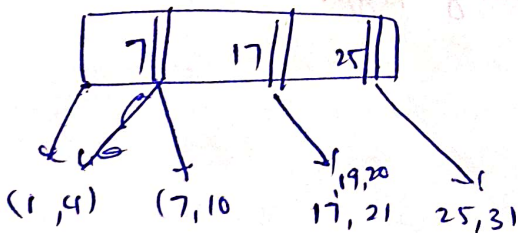
step 3:-



if we are trying to insert 21 leaf overflow will occur.

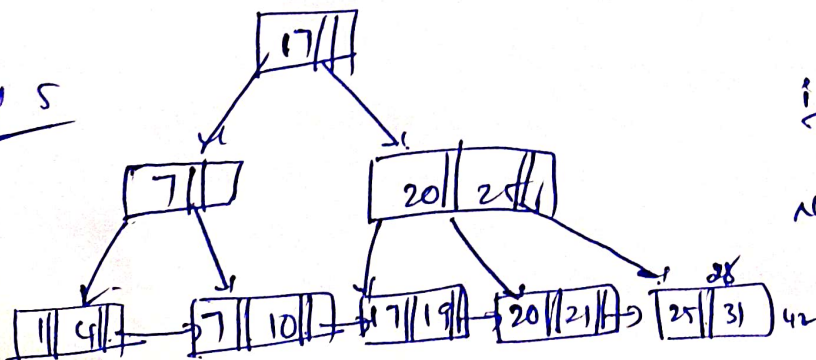
$(1, 4) \rightarrow (7, 10) \rightarrow (17, 21, 31)$

step 4



if we are trying to insert 25 leaf overflow will occur.

step 5



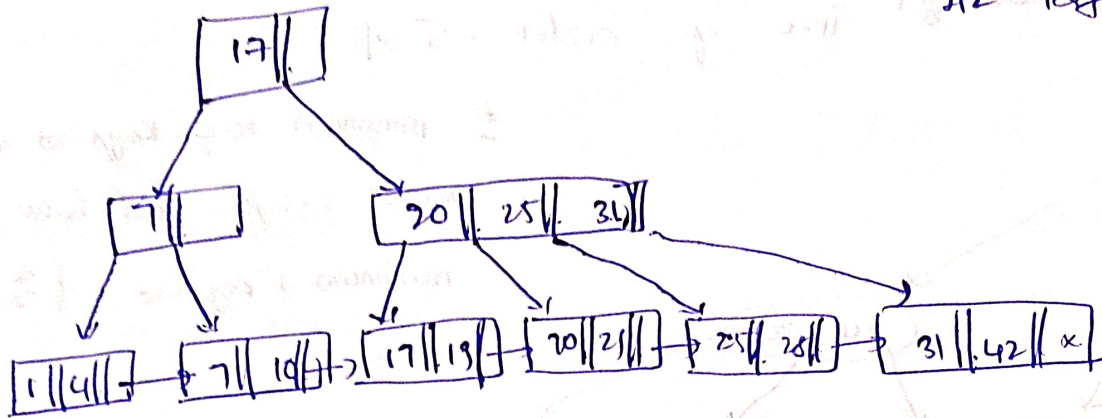
if we are trying to insert 20

non-leaf overflow will occur.

1st node contains $\lceil \frac{P}{2} \rceil - 1$

step 6

if we are trying to insert
12 leaf overflow will occur.



Deleting a Data Entry from B+ Trees

⇒ start at root, find leaf L where entry belongs

⇒ Remove the entry

⇒ if L contains at least $\lceil P/2 \rceil - 1$ entries, done!

⇒ if L has only $\lceil P/2 \rceil - 2$ entries

- Try to redistribute, borrowing from sibling
(adjacent node with same parent as L)

- if redistribution fails, merge L and a sibling

⇒ if merge occurred, then corresponding entry
from parent must be deleted

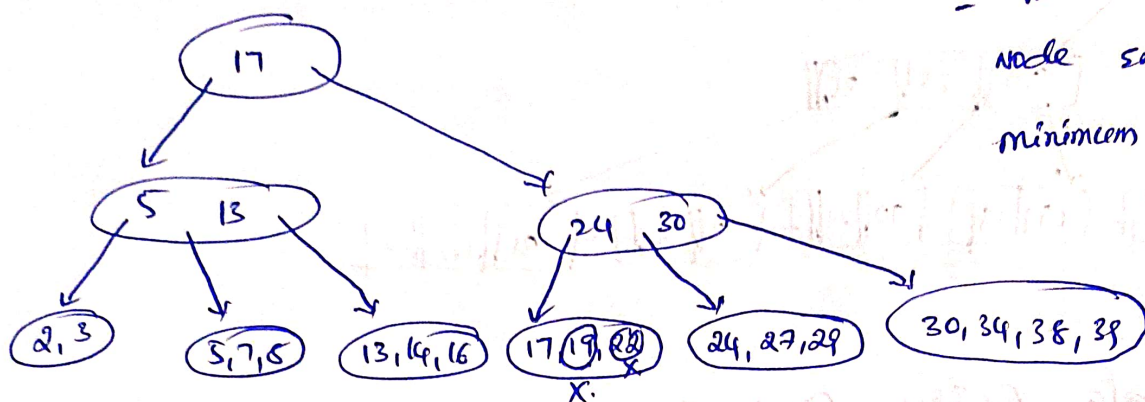
⇒ merge could propagate to root decreasing height

⇒ if the deleted entry present in internal node
replace it with inorder successor

Ex: delete 19, 22 from the given ~~B+ Tree~~ B+ Tree.

consider B+ Tree of order - 5 $\rightarrow$ P

* minimum no. of keys of any node except root have $\lceil P/2 \rceil$ - minimum 2 key need $\lceil \frac{5}{2} \rceil - 1 = 2$

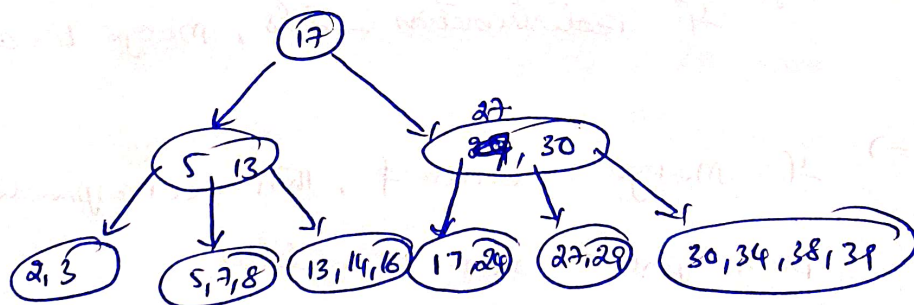


for deleting 19 $\rightarrow$ from root 17 $\rightarrow$ 24 $\rightarrow$ 17 $\rightarrow$ 19

delete 19, By deleting 19 check for underflow at least 2 keys
no need to merge $\rightarrow$ make rule 19 is not a part of anyone

for deleting 22:- search for '22' to reach leaf delete 22

By deleting '22' underflow occur, need to rearrange then
rearrange with sibling if it have enough number of keys



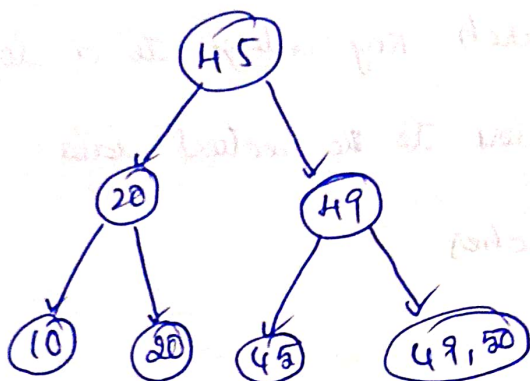
make rule '22' is not a part of any node

Ex 5

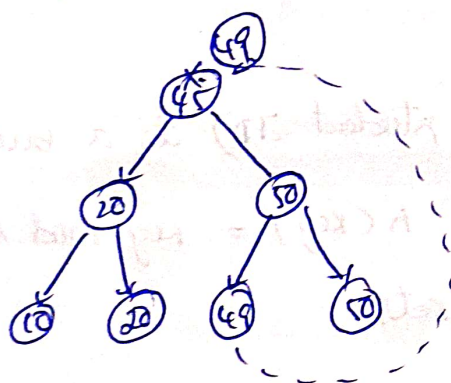
consider BT Tree of order - 3.

minimum each node have $\frac{P-1}{2} - 1$ keys except root

maxima $\frac{P-1}{2}$ key



delete 45 :- By deleting 45, underflow avoided, adjust keys by its siblings if it has enough number of keys



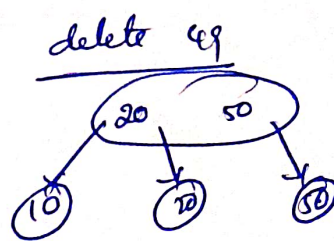
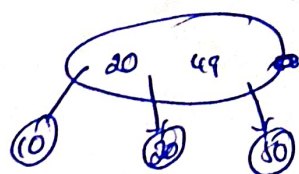
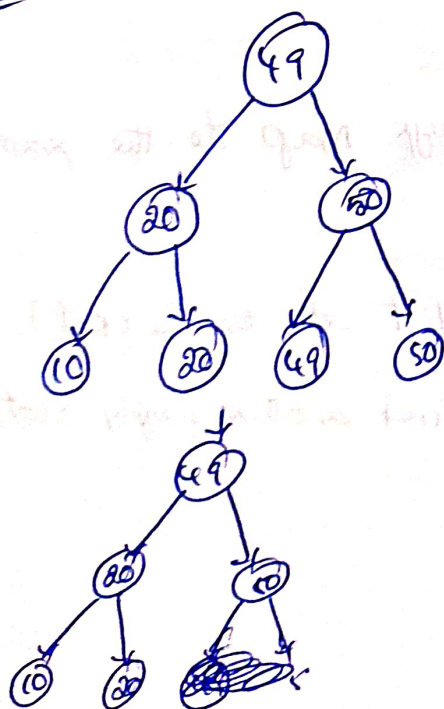
still 45 as root need to eliminate with smallest key from right subtree

now satisfies B+ tree property

=> delete 49

if we delete '49' underflow will occur, need to replace adjust with sibling But sibling don't have enough keys then need to merge. root (middle 50) need to pull down

=> ~~49~~ '50' try to adjust with '20' sibling need to merge automatically '49' pull down



Hash Based Indexing

Hash-Based index is a technique used to locate data quickly in a database using hash function.

The hash function maps search key values to a location in a hash table, which stores pointers to the actual data.

* fast for equality searches

where $id = 101$

* Not suitable for range queries

where $id > 100$ and $id < 200$.

Terminology :-

Hash function : Maps a key (like student ID) to a bucket (location) using a function like $h(\text{key}) = \text{key} \bmod N$,

where N is a number of buckets.

Bucket : stores pointers to the actual records.

collision : happens when two keys map to the same bucket.

* chaining (linked list at the bucket)

* open addressing (find another empty slot).

Hash Table

Index

Bucket (List of pointers to Records)

0

→ [Record 105, Record 205]

1

→ [Record 101]

2

→ [Record 102, Record 202]

Ex:-

create table students(

StudentID INT PRIMARY KEY,

Name VARCHAR2(10)

Branch VARCHAR2(10));

Assume we want to create hash index on StudentID.

Hash function:-

$$h(\text{StudentID}) = \text{StudentID} \% 5 \rightarrow (\text{5 Buckets} : 0 \text{ to } 4)$$

Insertion :-

<u>StudentID</u>	<u>Name</u>	<u>Branch</u>	<u>h(StudentID)</u>	<u>Bucket</u>
101	John	CSE	$101 \% 5 = 1$	1
102	Alice	ECE	$102 \% 5 = 2$	2
105	Bob	ME	$105 \% 5 = 0$	0
205	Carol	CSE	$205 \% 5 = 0$	0

Bucket structure :- (chaining used for collisions)

⇒ Bucket 0 → 105 → 205

Bucket 1 → 101

Bucket 2 → 102

Bucket 3 and 4 → Empty

Ex: select * from students where studentID = 205;

Hash function: $205 \% 5 = 0$

Go to Bucket 0

search linked list for 205 → Retrieve Record.

$(\text{node} \rightarrow \text{next}) \leftarrow \text{next of bucket} = (\text{studentID}) \% n$

Bucket	Hash Function	Bucket	Student ID
1	$101 \% 5 = 1$	205	101
2	$102 \% 5 = 2$	206	102
0	$105 \% 5 = 0$	207	103
0	$205 \% 5 = 0$	208	205